

Solution Overview

SLT Visual AI Agent

Field	Value
Project	SLT Mobitel AI-Powered Customer Support Agent
Version	1.1
Date	2026-05-17
Author	SLT AI Challenge Team VisionX

1. Executive Summary

The SLT Visual AI Agent is a production-ready, multi-channel intelligent customer support system built for SLT Mobitel — Sri Lanka's leading telecommunications provider. It combines large language model (LLM) reasoning, retrieval-augmented generation (RAG), real-time API integration via OpenAPI function calling, speech I/O, computer vision, webcam-based person detection, and human escalation into a unified conversational platform.

The system enables customers to get instant answers to telecom queries, check live account data (data balance, active plans, packages), request new services, and receive personalized support — across a web portal and WhatsApp — in English, Sinhala, and Tamil.

2. Business Context & Problem Statement

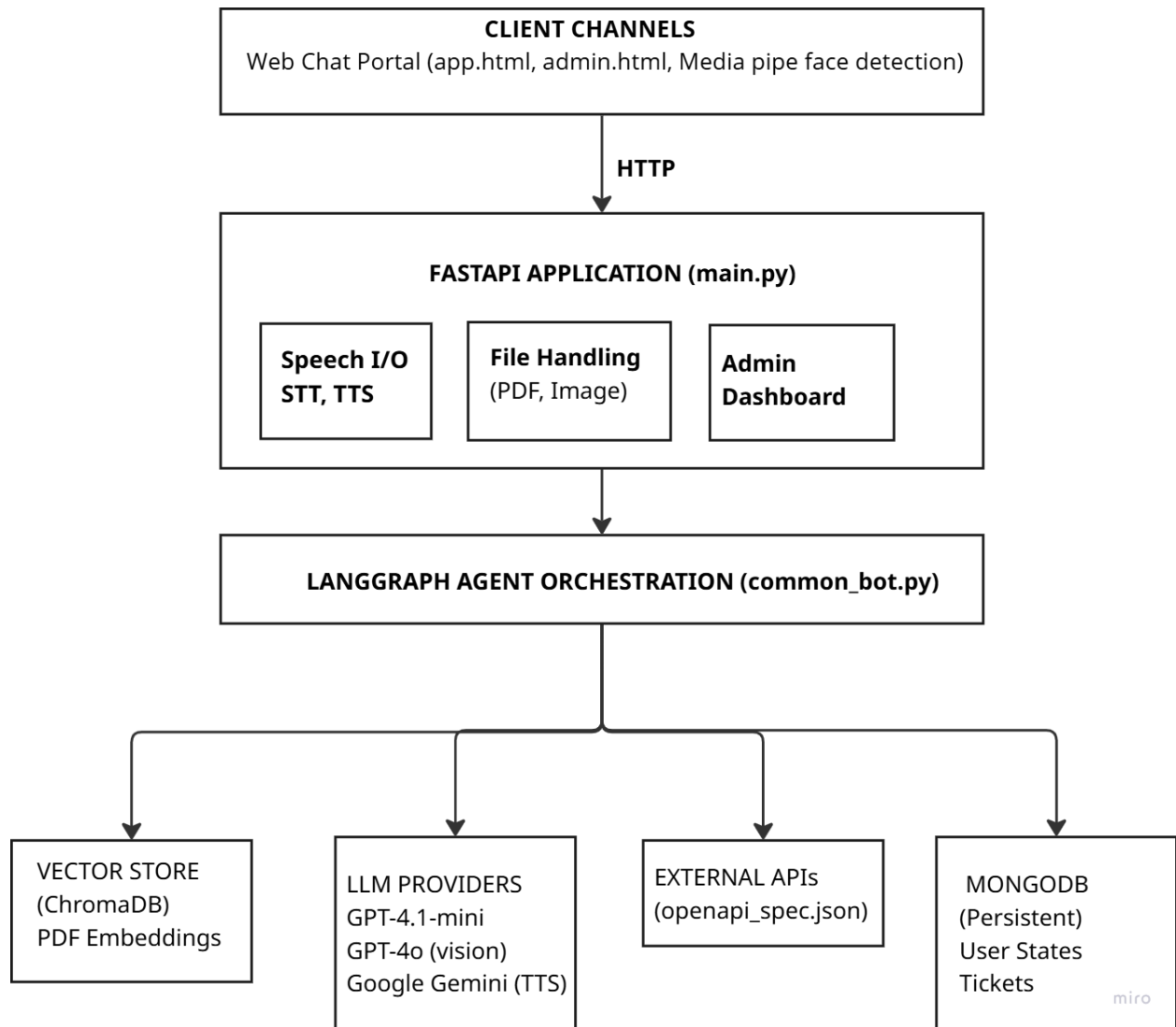
Customer support for a telecommunications company involves high query volumes spanning technical troubleshooting, service activations, billing inquiries, and product information. Traditional IVR systems and static FAQ pages fail to deliver personalized, context-aware responses.

Key pain points addressed:

- Long call center wait times for routine service requests
- Inability to handle multi-language queries at scale
- No self-service option for live account data (data balance, plan details, package management)
- No structured path for new service bookings (e.g., new broadband connections)
- Lack of a seamless handoff mechanism to human consultants when needed
- No unified knowledge base across documents and web content

3. Solution Architecture

3.1 High-Level Architecture



3.2 Technology Stack

Layer	Technology	Purpose
Web Framework	FastAPI 0.136	REST API server
Agent Orchestration	LangGraph 1.1	Multi-node state graph
LLM Abstraction	LangChain 1.2	Chain & prompt management
Primary LLM	OpenAI GPT-4.1-mini	Reasoning, classification, function calling
Vision LLM	OpenAI GPT-4o	Image analysis
Embeddings	OpenAI text-embedding-3-small	Document vectorization
Vector Database	ChromaDB 1.5	Semantic document search

Layer	Technology	Purpose
Database	MongoDB 7	State & data persistence
Speech-to-Text	Google Cloud Speech API	Audio transcription
Text-to-Speech	Google Gemini (Sinhala & Tamil)	Voice response synthesis
Web Scraping	Playwright	Website content extraction
PDF Processing	PyPDF2	Document text extraction
Face Detection	MediaPipe FaceDetector (Blaze Face)	Webcam-based presence detection
API Schema	OpenAPI 3.0 JSON	Backend API contract definition
Containerization	Docker + Docker Compose	Deployment

4. Core Components

4.1 FastAPI Application (main.py)

The central API layer handles all inbound requests and orchestrates the subsystems. It exposes endpoints for:

Chat & Conversation

- POST /common_chat/ — Primary chat endpoint; accepts text, voice, or file input
- GET /app — Serves the web chat UI
- POST /get_conversation — Retrieves full conversation history for a user

Knowledge Base Management

- POST /upload_pdf/ — Upload PDFs to the vector knowledge base (limit: 5 per org)
- POST /upload_website/ — Index a website URL into the knowledge base (limit: 5 per org)
- POST /list_pdf / POST /delete_pdf — PDF management
- POST /select_knowledge_base/ — Toggle between document and/or web context sources

API Specification Management *(new)*

- POST /upload_openapi_spec — Replace openapi_spec.json and hot-reload the API tools without a server restart
- GET /download_openapi_spec — Download the current openapi_spec.json for review or offline editing

Booking & Ticketing

- POST /get_bookings — Retrieve all service requests for an org

- POST /update_booking_status — Update a ticket's status (pending/confirmed/cancelled/completed)

Consultant Escalation

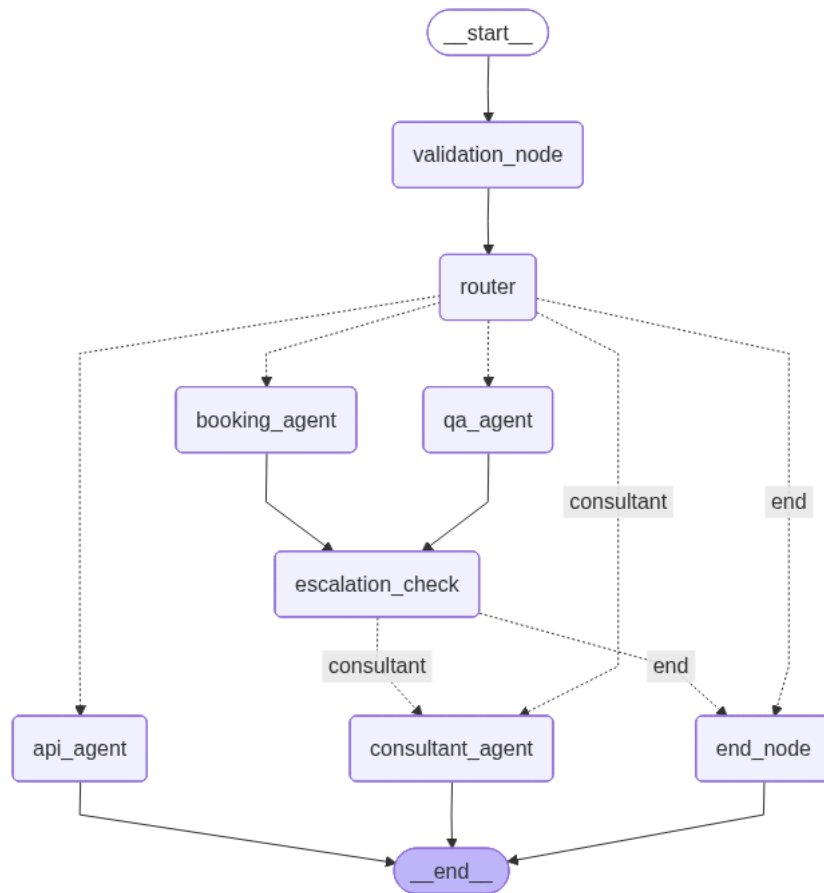
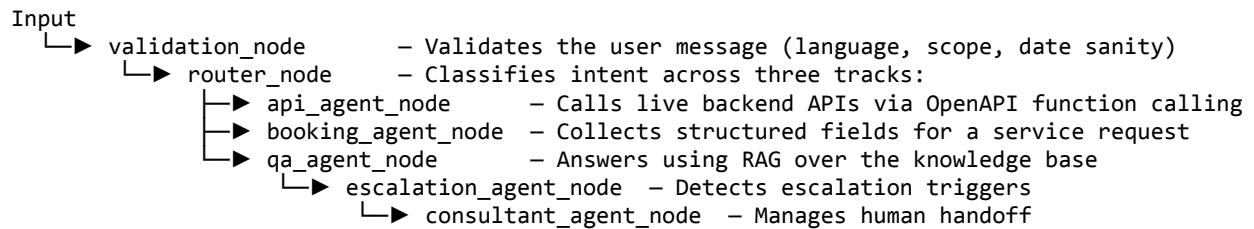
- POST /consultant_mode_switch — Enable or disable human handoff for a user session
- POST /consultant_send_message — Allow a human consultant to inject a message

Admin & Configuration

- GET /admin — Serves the admin dashboard UI
- POST /set_tts — Enable or disable text-to-speech responses

4.2 LangGraph Agent (common_bot.py)

The conversational engine is a directed state graph with six nodes:



Routing priority in router_node:

1. If `api_session.pending_call` exists → continue to `api_agent` (multi-turn param collection in progress)
2. Else if `detect_api_call()` returns true → route to `api_agent`
3. Else if `detect_general_question()` returns true → route to `qa_agent`
4. Else → route to `booking_agent`

State tracked per user session:

- Full conversation history
- Active service bookings and collected fields
- API session state (`api_session`) with pending call details
- Escalation and consultant mode status
- Timestamp for freshness validation

4.3 Retrieval-Augmented Generation (RAG)

When a user asks a general question, the QA node retrieves relevant context before generating an answer:

1. Uploaded PDFs are extracted with PyPDF2, chunked (500-character segments with overlap), and embedded using OpenAI's text-embedding-3-small model into ChromaDB — partitioned by `org_id`
2. Websites are scraped with Playwright, text is extracted and stored in MongoDB
3. At query time, semantic search retrieves the top-k relevant chunks
4. The LLM generates an answer grounded in the retrieved context, reducing hallucination

4.4 API Integration Node (`utils/api_node.py`) (new)

A dedicated LangGraph node that allows the bot to call live backend APIs defined in a standard OpenAPI 3.0 JSON specification. This gives the bot access to real-time account data without hardcoding any API logic into the agent.

How It Works**1. Spec loading**

`openapi_spec.json` is parsed at server startup and converted into OpenAI function-calling tool schemas via `openapi_to_tools()`. Each API path becomes a callable tool with typed parameters derived directly from the spec's parameters and `requestBody` fields.

2. API identification via function calling

When routed to `api_agent_node`, the LLM is invoked with `llm.bind_tools(openai_tools)`. It selects the correct operation (e.g., `getDataBalance`) and extracts whatever parameter values are already present in the conversation.

3. Multi-turn parameter collection

If required parameters are missing (e.g., `customer_id` not yet provided), the node:

- Stores the pending call in state['api_session']['pending_call']
- Asks the user for the missing information in plain language (field descriptions from the spec are used, not raw parameter names)
- On the next turn, router_node detects the pending call and routes straight back to api_agent_node
- Newly provided values are extracted and merged into collected_params
- This repeats until all required parameters are gathered

4. API execution

Once all parameters are collected, simulate_api_call() is invoked. In the current development phase this returns realistic dummy responses. In production, this function is replaced with real HTTP calls to the server specified in servers[0].url of the spec.

5. Response formatting

The raw API result is passed to an LLM formatting step that presents the data to the user in a friendly, conversational way — no JSON or technical terms are exposed.

Currently Available API Operations (Dummy APIs for Demo)

Operation ID	Method	Endpoint	Description
getDataBalance	GET	/user/{id}/databalance	Remaining data balance across all active packages
activatePackage	POST	/user/{id}/activate_package	Activate a data or voice package
cancelPackage	POST	/user/{id}/cancel_package	Cancel an active package
getCurrentPlan	GET	/user/{id}/current_plan	Current broadband/mobile plan details

Hot-Reload via Admin UI

The spec can be replaced at runtime via the admin dashboard without restarting the server. Uploading a new openapi_spec.json triggers reload_spec(), which rebuilds openai_tools in memory immediately — new operations become available on the very next chat turn.

4.5 Booking Agent

For service requests (e.g., new broadband connections, technical support), the booking agent collects required information progressively:

- Services are defined with required fields in utils/prompts.py
- The agent identifies which fields are still missing and asks targeted follow-up questions
- When all required fields are collected, a quotation is automatically generated
- The completed booking is persisted in MongoDB and surfaced in the admin dashboard

Supported services:

Service Key	Description
telco_support	Technical support request routing
new_connection	Broadband/fiber new connection intake

4.6 Speech Processing

Input (Speech-to-Text):

Audio uploaded by users (WebM, OGG, MP3, FLAC, WAV) is transcribed using Google Cloud Speech-to-Text API with support for:

- en-US — English
- si-LK — Sinhala
- ta-IN — Tamil

Output (Text-to-Speech):

When TTS is enabled, bot responses are synthesized to MP3 audio files using either Google Gemini or Anthropic Claude as the TTS provider. Audio URLs are returned alongside text responses.

4.7 Vision Capability

When a user uploads an image, the system sends it to OpenAI GPT-4o for visual analysis. The model's description of the image is injected into the conversation context, enabling the agent to reason about visual content (e.g., a screenshot of an error, a router light status, a bill scan).

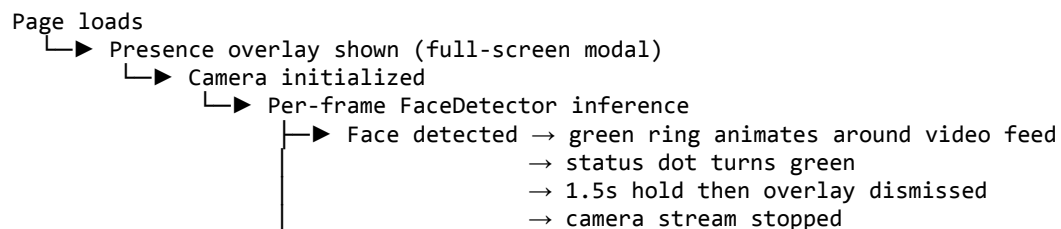
4.8 Webcam-Based Person Detection (*new*)

The web chat portal includes a client-side real-time person presence detection system powered by **MediaPipe FaceDetector** (Blaze Face Short Range model). This runs entirely in the browser with no server-side processing.

Implementation

- **Model:** blaze_face_short_range.tflite loaded via the MediaPipe Tasks Vision CDN (v0.10.14)
- **Camera access:** navigator.mediaDevices.getUserMedia({ video: true }) at 640×480
- **Inference:** Runs on requestAnimationFrame() — continuous detection with GPU acceleration (falls back to CPU)
- **Confidence threshold:** minDetectionConfidence: 0.5

Detection Flow



- ↳ No face → ring hidden, dot turns gray
 - 5-second lockout if face previously seen

UI Elements

Element	ID	Purpose
Full-screen overlay	presence-overlay	Blocks UI until presence confirmed
Video feed	presence-video	Live camera stream
Detection ring	face-ring	Green animated ring when face detected
Status dot	presence-dot	Header indicator (green/gray)
Skip button	—	Allows bypassing detection without camera

The 5-second lockout timer (LOCKOUT_MS = 5000) prevents re-authentication abuse: if a detected face leaves the frame, the user must wait before the overlay reappears.

4.9 WhatsApp Integration

The system integrates with the Meta WhatsApp Cloud API to:

- Send a consultant escalation notification to the consultant

4.10 Human Escalation (Consultant Mode)

When the escalation agent detects triggers (e.g., user frustration, complex issue, explicit request for a human), it:

1. Notifies the user that a consultant will assist
2. Switches the user session to consultant mode
3. Suppresses bot responses and surfaces the conversation in the admin portal
4. Allows a human consultant to type replies directly into the session
5. Can be toggled off to return control to the bot

5. Data Model

MongoDB Collections

Collection	Key Fields	Purpose
user_states	org_id, user_id, chat_history, consultant_mode, escalation_status, api_session	Session, conversation, and API call state

Collection	Key Fields	Purpose
user_bookings	org_id, user_id, service_key, collected_info, booking_status	Service request records
pdf_documents	org_id, document_name, file_size	PDF metadata registry
site_data	org_id, url, content, indexed_at	Scraped website content
organization	org_id, selected_knowledge_bases	Org-level settings
unread_counts	org_id, user_id, unread_count	Portal unread message tracking

ChromaDB

PDF embeddings are stored in a persistent ChromaDB instance, namespaced by org_id to enforce multi-tenant data isolation.

openapi_spec.json

A standard OpenAPI 3.0 JSON file at the project root that defines all callable backend APIs. This file is the single source of truth for:

- Which operations the api_agent_node can invoke
- Parameter names, types, descriptions, and required/optional status
- Server base URL for production HTTP calls
- Admin-replaceable at runtime via POST /upload_openapi_spec

6. Multi-Tenancy

The system supports multiple organizations (org_id) on a single deployment:

- All MongoDB collections are partitioned by org_id
- ChromaDB vector collections are namespaced per org
- Knowledge base sources, TTS settings, and consultant assignments are configured per org
- Document and website limits are enforced per org (default: 5 each)

7. User Interfaces

7.1 Web Chat Portal (app.html)

A single-page application providing:

- **Webcam presence detection** on page load — user must be in frame before chat is accessible
- User list sidebar with unread message badges
- Conversation view with sent/received message bubbles

- Text input with audio recording and file attachment buttons
- Consultant override input area for human agents
- Real-time TTS audio playback on bot responses

7.2 Admin Dashboard (admin.html)

A tabbed management interface covering:

Tickets Tab

- Table of all service requests with status (pending / confirmed / cancelled / completed)
- Status update controls
- Summary statistics cards

Knowledge Base Tab

- PDF upload (with size/count validation)
- Website URL indexing
- List and delete existing sources
- Toggle between doc-only, web-only, or both context modes
- **API Specification card (new)** — upload a new openapi_spec.json to update available API tools with immediate hot-reload; download the current spec for offline editing

Settings Tab

- Org ID configuration
- Knowledge base source selection
- TTS enable/disable toggle

8. Deployment

Docker Compose

The application is packaged as two Docker services:

Service	Image	Port	Purpose
app	Custom FastAPI image	5001	Application server
mongodb	mongo:7	27017	Database

Persistent Docker volumes are used for:

- chat_uploads/ — User-uploaded files
- audio_tts/ — Generated TTS audio
- avatar_videos/ — Avatar video assets
- chroma_store/ — Vector database persistence

- logs/ — Application logs

MongoDB health checks ensure the database is ready before the application starts.

Environment Configuration

All secrets and configuration are managed via .env:

- OPENAI_API_KEY — OpenAI API credentials
- GEMINI_API_KEY — Google Gemini credentials
- GOOGLE_APPLICATION_CREDENTIALS — GCP service account JSON path
- ACCESS_TOKEN, PHONE_NUMBER_ID — WhatsApp Cloud API credentials
- DB_HOST, DB_PORT, DATABASE — MongoDB connection
- KB — Active knowledge base sources (["doc", "web"])
- AGENTS — Consultant phone numbers (WhatsApp)
- USE_TTS, TTS_PROVIDER — Voice output configuration
- TIME_ZONE — Server timezone (default: Asia/Colombo)

9. Security Considerations

- API keys and service credentials are stored in environment variables, not in source code
- Multi-tenant data isolation is enforced at the application layer via org_id namespacing
- File upload size limits and count limits are enforced to prevent resource abuse
- WhatsApp webhooks use Meta's verification token pattern
- Uploaded openapi_spec.json is validated as parseable JSON before being accepted
- Google Cloud service account credentials are stored as a JSON key file (should be mounted as a secret in production)
- MediaPipe face detection runs entirely client-side — no biometric data leaves the browser

10. Key Workflows

General Q&A Flow

```
User sends question
→ STT transcription (if audio)
→ Validation node
→ Router: classified as general question
→ Knowledge classifier: RAG needed?
  → Yes: Retrieve PDF + web chunks from ChromaDB / MongoDB
  → Construct grounded prompt
→ LLM generates answer
→ TTS synthesis (if enabled)
→ Response returned (text + audio URL)
→ Chat history saved to MongoDB
```

API Call Flow

User: "What's my data balance?"

- Validation node
- Router: detect_api_call() → TRUE → api_agent_node
- LLM function calling identifies: getDataBalance
- Extracted params: {} (none found)
- Missing required: [customer_id]
- Ask user: "Could you share your customer account ID?"

User: "My ID is CUS-12345"

- Router: api_session.pending_call exists → api_agent_node
- extract_pending_params() → {customer_id: "CUS-12345"}
- All params collected → simulate_api_call("getDataBalance", {...})
- format_api_response() → friendly natural-language summary
- Response returned; api_session.pending_call cleared
- State saved to MongoDB

Service Booking Flow

User requests a service (e.g., "I want a new fiber connection")

- Router: classified as booking request
- Booking agent: identify service type
- Check MongoDB for already-collected fields
- Identify missing required fields
- Ask user for next missing field
- Repeat until all fields collected
- Generate quotation
- Save booking to MongoDB with status: pending
- Admin portal displays new ticket

Consultant Escalation Flow

Escalation node detects trigger (frustration / complexity / explicit request)

- Notify user: consultant will assist
- Set consultant_mode = true in MongoDB
- Bot suppresses automatic responses
- Admin portal highlights the user session
- Human consultant types reply via admin UI
- Message delivered to user
- Consultant toggles off → bot resumes control

API Spec Update Flow

Admin opens Knowledge Base tab in admin.html

- Selects new openapi_spec.json file
- Clicks "Upload & Reload Spec"
- POST /upload_openapi_spec
 - File validated as JSON
 - Written to openapi_spec.json on disk
 - reload_spec() called in-process:
 - openapi_spec updated
 - openai_tools rebuilt from new spec
- Next chat turn uses new API definitions immediately
- Admin can download current spec via GET /download_openapi_spec

11. Limitations & Future Considerations

Area	Current State	Potential Improvement
Languages	English, Sinhala, Tamil	Extend with additional regional languages
Authentication	No user authentication layer	Add OAuth2 / API key authentication for production
API Node	Simulated responses (dummy mode)	Replace simulate_api_call() with real HTTP calls using the spec's servers[0].url
API Security	No auth on external API calls	Add bearer token / API key injection per spec securitySchemes
Person Detection	Client-side face presence only	Server-side identity verification, liveness detection
Avatar Video	Used a real-time working avatar for a third-party, with a readymade avatar	More accurate lip-sync and SLT-specific avatar rig
Monitoring	File-based logging	Integrate structured logging with Datadog / CloudWatch
Scalability	Single FastAPI instance, yet it can serve multiple users	Add a load balancer and horizontally scale app containers

12. Summary

The SLT Visual AI Agent is a comprehensive, production-deployable AI customer support platform that demonstrates the following capabilities in a single integrated system:

- **Conversational AI** with multi-turn memory and structured intent routing via LangGraph
- **Live API integration** — OpenAPI spec-driven function calling lets the bot retrieve real-time account data and perform account actions without hardcoded logic
- **RAG** over organizational documents and websites for grounded, accurate answers
- **Multimodal input** supporting text, audio (3 languages), images, and PDFs
- **Structured booking workflows** with automatic field collection and quotation generation
- **Human-in-the-loop escalation** with seamless consultant handoff
- **Webcam person detection** using MediaPipe FaceDetector for client-side presence verification before chat access
- **Multi-channel delivery** via web portal and WhatsApp
- **Multi-tenant architecture** supporting multiple organizations on one deployment
- **Admin tooling** for knowledge base management, API spec hot-reload, ticket tracking, and configuration